

Optimized NTT Architecture Based on the Plantard Algorithm for ML-KEM and ML-DSA

Jiansheng Chen*, Bei Wang*, Ziyang Ni, *Member, IEEE*, Mengxue Li, Fei Lyu, Yijun Cui, *Member, IEEE*, and Weiqiang Liu, *Senior Member, IEEE*

Abstract—Modular multiplication is a vital operation in the Number Theoretic Transform (NTT), significantly enhancing polynomial multiplication in Post-Quantum Cryptography (PQC). The design efficiency of modular multiplication directly influences the computational performance of polynomial computation units. This work marks the first hardware-oriented improvement of the Plantard algorithm, optimizing the NTT architecture. We modify the Plantard algorithm and propose three innovative enhanced versions tailored for lattice-based cryptography (LBC). By employing pre-processed twiddle factors for result correction and eliminating an additional constant multiplication, we greatly simplify the computation steps. Based on these improvements, we further design a lightweight BRAM-free iterative NTT and a high-speed Multi-path Delay Commutator (MDC) pipelined NTT, both targeting the ML-KEM and ML-DSA parameter sets. Implementation results on the Xilinx Artix-7 platform demonstrate that our Plantard_{prew} design reduces slice usage by 22% to 53% and delay by 41.1% to 44.3% compared to existing algorithms like Barrett and K²RED. Furthermore, our iterative NTT design achieves the minimal area-time product (ATP) among state-of-the-art implementations, with reductions of 61.3% and 40.8% in ENS, and frequency increases of 72.7% and 145.4% for ML-KEM and ML-DSA, respectively. The pipelined NTT design also reduces delay by 23.7% and ATP by 4.9%, showcasing the compactness and superior performance of our approach.

Index Terms—ML-KEM, ML-DSA, NTT, Modular reduction, Hardware Implementation, FPGA

I. INTRODUCTION

The US National Institute of Standards and Technology (NIST) standardization process aims to select cryptographic algorithms that are still secure after the advent of quantum computers. As of August 2024, NIST published three Federal Information Processing Standards (FIPS) for Post-Quantum Cryptography (PQC): FIPS-203, which is called the lattice-based public key encryption algorithm (ML-KEM [1], based on CRYSTALS-Kyber); FIPS-204, which is called the digital signature algorithm (ML-DSA [2], based on CRYSTALS-Dilithium); and FIPS205, which is called the hash-based digi-

tal signature algorithm (SLH-DSA [3], based on SPHINCS+). Most of the current standardized PQC schemes are lattice-based cryptographic schemes (LBCs). Compared with other schemes, LBC schemes are more suitable for practical applications due to the small size of the public key and private key, fast computation speed, and the ability to be used for constructing various cryptographic primitives.

Polynomial multiplication is one of the key operations in LBC schemes. It can be performed using the traditional schoolbook algorithm ($O(n^2)$) or accelerated using the Number Theoretic Transform (NTT) algorithm ($O(n \log n)$). Therefore, in PQC schemes, many LBC schemes adopt NTT-friendly primes to provide NTT acceleration, such as ML-KEM and ML-DSA. Whether in software or hardware, designing an NTT generally involves addressing the following two questions:

- 1) Which type of NTT? Pipeline or Iterative?
- 2) Which modular reduction algorithm should be used?

The current NTT designs mainly include two types: high-speed pipelined NTT and low-resource iterative NTT. Pipelined NTT utilizes multi-stage butterfly units and multiplexing to unfold the NTT process into multi-stage pipelined operations, allowing for efficient completion of NTT/INTT in fewer cycles at the cost of higher resource overhead, such as [4], [5], [6]. Iterative NTT generally requires additional storage units to temporarily hold the intermediate values of each butterfly operation, allowing the NTT computation to be completed with fewer butterfly units, such as [7], [8], [9], [10]. Typically, if higher performance is desired, a pipelined NTT is the preferred choice for its lower latency and higher throughput. In contrast, an iterative NTT is more suitable for scenarios with limited resources. In addition to the NTT designs mentioned above, some designs focus on optimizing the NTT architecture to enhance the flexibility of NTT units and implement a more efficient storage scheme. In 2022, Chen *et al.* proposed a scalable radix-2/4 NTT with efficient conflict-free memory mapping [11], which enhances the configurability and computational efficiency of NTT units. More recently, a BRAM-free NTT architecture is proposed in 2023 [12], making it more suitable for resource-constrained devices.

Modular multiplication, which includes both multiplication and modular reduction, is a key computational unit in NTT and has a significant impact on the overall operating frequency and cycles. For primes of ML-KEM and ML-DSA, in 2021, Xing *et al.* proposed an improved Barrett modular reduction [7] and applied it in the pure hardware implementation of the overall ML-KEM system, achieving a more compact ML-

*Jiansheng Chen and Bei Wang contributed equally to this work. (Corresponding authors: Weiqiang Liu and Yijun Cui.)

This work was supported in part by the National Natural Science Foundation of China under Grant 62474092, Grant 62371226, Grant 62425404, Grant 62022041, Grant 62571233, Grant 62201234 and Grant 62134002; and in part by the Postgraduate Research & Practice Innovation Program of Jiangsu Province under Grant KYCX25_0644.

Jiansheng Chen, Bei Wang, Mengxue Li, Fei Lyu, Yijun Cui, and Weiqiang Liu are with the College of Integrated Circuits, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China (e-mail: {cjs1123, wangbei91, limengxue, lyufei, yijun.cui, liuweiqiang}@nuaa.edu.cn).

Ziyang Ni is with the Centre for Secure Information Technologies (CSIT), Queen's University Belfast, Belfast BT3 9DT, U.K. (e-mail: z.ni@qub.ac.uk).

KEM design on Xilinx Artix-7 device. In the same year, based on the KRED modular reduction algorithm [13], Bisheh-Niasar *et al.* proposed optimized K²RED design [8]. This architecture limits the results to 12 bits by performing the KRED operation twice. Zhang *et al.* proposed a LUT-based modular reduction design using carry save adder (CSA) and lookup tables (LUTs) [9]. It is proposed with resources and critical path well optimized and thus has a better performance compared to other designs. Yaman *et al.* recursively exploited the property $2^{12} \equiv 2^9 + 2^8 - 1 \pmod{3329}$ to progressively reduce operands from 12 bits to fewer than 12 bits [10], completing the modular reduction. In addition, Ni *et al.* proposed a modular reduction algorithm [12] combining the KRED algorithm and LUT method based on [9] in 2023, which has less hardware overhead and higher maximum frequency than the original Barrett algorithm. Moreover, Plantard *et al.* proposed the Plantard algorithm in 2020, an efficient modular reduction method specifically tailored for applications with operand sizes smaller than a word size (*i.e.*, 32/64 bits) [14]. In 2022, Huang *et al.* proposed an improved version of the Plantard algorithm [15] based on Cortex-M4 platform that adapts to signed inputs, increasing the input range and reducing the output range. Can *et al.* completed the hardware implementation and comparison of the K²RED and Plantard algorithms [16]. The Plantard algorithm still has significant optimization potential in pure hardware design.

Existing NTT implementations utilize various modular reduction algorithms and diverse architectural designs, lacking unified benchmarks or standardized frameworks. The current NTT implementations still lack a design that effectively balances resource efficiency and computational performance. In this article, we delve into the optimization of the Plantard multiplication algorithm from different perspectives and two types of NTT are implemented to evaluate performance and resource usage. It is worth noting that this is the first work focused on hardware improvements for the Plantard algorithm and the optimization designs of two NTT architectures in ML-KEM/ML-DSA. The main contributions of this paper are as follows:

- Three novel enhanced Plantard modular reduction algorithms are proposed: ShiftAdd-Plantard, Plantard_preR, and Plantard_prew, each optimized for different application scenarios. We improve the original Plantard algorithm by applying grouped partial product summation, merging pre-processing and constant multiplication into the twiddle factors, and adjusting the computation process. As a result, our modular multiplication units based on ShiftAdd-Plantard and Plantard_prew achieve significant improvements in both resource efficiency and operating frequency. Notably, the Plantard_prew unit completes its operation in just four cycles and reaches a frequency of 499 MHz, demonstrating a thoroughly optimized design from algorithm to hardware implementation.
- We design a lightweight iterative NTT architecture without BRAM, applying the Plantard_prew algorithm for ML-KEM and the ShiftAdd-Plantard algorithm for ML-DSA. This architecture features interleaved access to

three FIFOs for butterfly operations, enabling flexible data flow and reduced area. Dynamic switching between NTT and INTT is supported through signal-controlled Cooley–Tukey and Gentleman–Sande units, while a dual-butterfly structure enhances throughput and simplifies control logic, achieving near-theoretical cycle performance for both schemes.

- We also propose a pipelined MDC NTT architecture based on Plantard_prew and ShiftAdd-Plantard for ML-KEM and ML-DSA. This design optimizes data delays and butterfly group lengths to improve efficiency and reconfigurability. By separating storage and computation units and centralizing storage management, it reduces FPGA resource usage and shift register overhead, enabling flexible and resource-efficient NTT/INTT operations suitable for both ML-KEM and ML-DSA.
- We present a detailed hardware evaluation of modular reduction and NTT designs. Our BRAM-free iterative NTT completes operations in 457/1034 cycles, using only 803/2038 and 895/1452 FFs and 2/4 DSPs for ML-KEM and ML-DSA, achieving the lowest ENS and ATP among comparable designs. The pipelined version for ML-DSA runs at over 200 MHz on Artix-7 and completes in 169/335 cycles, reducing delay by 23.7% and improving ATP by 4.9% over state-of-the-art. This work bridges theory and FPGA implementation, providing practical insights for optimizing performance and resource usage.

This paper is organized as follows. Section II introduces NTT, modular multiplication techniques, and the Plantard algorithm. Section III presents three improved Plantard algorithms, along with related proofs and their hardware implementation. Optimized Plantard modular multiplication units and two NTT architectures are described in Section IV. Section V provides implementation results and a comparison with previous designs, and Section VI concludes the paper.

II. PRELIMINARIES

A. Number Theoretic Transform

ML-KEM and ML-DSA schemes operate on the polynomial ring $R_q = \mathbb{Z}_q/(x^n + 1)$, its coefficients have to be modulo q (*i.e.*, these lie in the range $[0, q - 1]$) and the degree of the polynomial is less than n ($n = 256$). The NTT eliminates the complex floating-point and multiplication-addition calculations present in the Fast Fourier Transform (FFT) by performing all operations within a finite field. It uses integer roots of unity, rather than the complex roots of unity used in the FFT (collectively referred to as twiddle factors w). For a polynomial $a(x) = \sum_{i=0}^{n-1} a_i x^i \in R_q$, NTT transforms the coefficient vector $\mathbf{a} = (a_0, a_1, \dots, a_{n-1})$ into the frequency-domain vector $\hat{\mathbf{a}} = (\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{n-1})$.

For ML-KEM, $q = 3329$ with ω a 256-th root of unity, the polynomial $x^{256} + 1$ can be factored into 128 polynomials of degree 2 modulo q , *i.e.*, $x^{256} + 1 = \prod_{i=0}^{127} (x^2 - \omega^{2i+1})$. Therefore, there is an incomplete NTT in ML-KEM, divided

into odd and even parts:

$$\begin{aligned}\hat{a}_{2i} &= \sum_{j=0}^{127} a_{2j} \omega^{(2\text{br}_7(i)+1)j}, \\ \hat{a}_{2i+1} &= \sum_{j=0}^{127} a_{2j+1} \omega^{(2\text{br}_7(i)+1)j}\end{aligned}\quad (1)$$

Here, $\text{br}_7(i)$ for $i = 0, 1, \dots, 127$ represents the bit inversion of an unsigned 7-bit integer i . The INTT is described as follows:

$$\begin{aligned}a_{2i} &= \frac{1}{128} \sum_{j=0}^{127} \hat{a}_{2j} \omega^{-(2\text{br}_7(i)+1)j}, \\ a_{2i+1} &= \frac{1}{128} \sum_{j=0}^{127} \hat{a}_{2j+1} \omega^{-(2\text{br}_7(i)+1)j}\end{aligned}\quad (2)$$

For ML-DSA, the modulo $q = 8380417$ is chosen such that there exists a 512-th root of unity ω modulo q . This implies that the polynomial $x^{256} + 1$ splits into $x^{256} + 1 = \prod_{i=0}^{255} (x - \omega^{2i+1})$. Then the negative wrapped convolution technique to avoid doubling the input size of NTT can be applied:

$$\hat{a}_i = \sum_{j=0}^{255} a_j \omega^{(2\text{br}_8(i)+1)j} \bmod q \quad (3)$$

Here, $\text{br}_8(i)$ for $i = 0, 1, \dots, 255$ represents the bit inversion of an unsigned 8-bit integer i . Also, INTT is described as follows:

$$a_i = \frac{1}{256} \sum_{j=0}^{255} \hat{a}_j \omega^{-(2\text{br}_8(i)+1)j} \bmod q \quad (4)$$

When using NTT to accelerate the polynomial multiplication $c(x) = a(x) \cdot b(x)$, the process involves several key steps. First, apply Eq. 1 and Eq. 3 to convert polynomials $a(x)$ and $b(x)$ to the NTT domain. Then, perform point-wise multiplication (PWM) by $\hat{c}(x) = \hat{a}(x) \circ \hat{b}(x)$, and finally convert the NTT domain polynomial $\hat{c}(x)$ back to the normal domain using Eq. 2 and Eq. 4.

B. Modular Multiplications

Since polynomial multiplication is performed modulo q , all operations in NTT are modular. In software implementation, modular reduction replaces division with constant-time multiplication and shifts [15]. In hardware implementation, while modular addition and subtraction are straightforward, modular multiplication is challenging due to division and modulus operations. Thus, efficient modular reduction is essential for both hardware and software platforms. Generally, modular multiplication consists of three steps:

- 1) **Integer multiplication**
- 2) **Modular reduction**
- 3) **Results Correction (if necessary)**

Currently, in most NTT designs, multiplication is carried out by DSPs in FPGA, while modular reduction is performed using various algorithms. The performance of the modular multiplication unit, which involves the critical path and resource

usage, is mainly influenced by modular reduction. Therefore, an efficient NTT architecture relies on the optimized design of a modular multiplication unit with low resource and low latency.

C. Modular Reduction Algorithm

The essence of modular reduction algorithms is to efficiently compute $x \bmod q$ in order to optimize computation, control numerical range, and ensure mathematical equivalence. Commonly used modular reduction algorithms in NTT include Montgomery-based, Barrett-based, and $K^2\text{RED}$ -based approaches. A comparison of these three algorithms is presented in Table I. For detailed descriptions of these algorithms, please refer to [8], [17], [18].

Among the three modular reduction algorithms, Montgomery, Barrett, and $K^2\text{RED}$, only $K^2\text{RED}$ requires a specific modulus form (i.e., $q = k \cdot 2^m + 1$), while both Montgomery and Barrett are applicable to general moduli. In the context of LBC, the Montgomery method is widely adopted in software due to its efficiency in repeated modular multiplications. However, in hardware implementations, it is gradually being replaced because of its additional pre/post-processing overhead. Instead, Barrett and $K^2\text{RED}$ are favored in hardware, particularly in NTT architectures, where $K^2\text{RED}$ is especially effective when the modulus q satisfies its structural requirements.

D. Plantard Algorithm

Compared to other modular multiplication algorithms, Plantard is specifically designed to enhance the speed of NTT when the modulus is relatively small (a few dozen bits). The process of the Plantard algorithm is shown in Algorithm 1. It is important to note that the condition q must be an odd modulus and satisfy $q < \frac{2^n}{\varphi}$, where $\varphi = \frac{1+\sqrt{5}}{2}$ [14]. For the moduli used in the CRYSTALS-ML-KEM and CRYSTALS-ML-DSA, where $q = 3329$ and $q = 8380417$, we determine that the smallest values for n are 13 and 24, respectively. For more detailed about proofs for the Plantard algorithm, please refer to [14].

Algorithm 1 Plantard Algorithm [14]

Input: A, B, R, n, q with $0 \leq A, B < q, R = q^{-1} \bmod 2^{2n}$

Output: $C = AB(-2^{-2n}) \bmod q, 0 \leq C < q$

- 1: $C_0 \leftarrow ABR \bmod 2^{2n}$
 - 2: $C_1 \leftarrow \left\lfloor \frac{C_0}{2^n} \right\rfloor + 1$
 - 3: $C \leftarrow \left\lfloor \frac{C_1 q}{2^n} \right\rfloor$
 - 4: **if** $C = q$ **then**
 - 5: **return** 0
 - 6: **end if**
 - 7: **return** C
-

According to the algorithm process, in addition to multiplying operands, the Plantard algorithm mainly includes several key operations: multiplying by the constant R and q , left-shifting by n bits, extracting the least significant $2n$ bits, and

TABLE I
COMPARISON OF MODULAR REDUCTION ALGORITHMS

Algorithm	Requirements	Pre/Post Processing	Output Format	Best Use Case	Performance
Montgomery [17]	In Montgomery domain	Required	Montgomery domain	Repeated modular multiplications	Fast for repeated ops
Barrett [18]	Input $< q^2$	None	Standard integers	General-purpose use	Moderate
K ² RED [8]	$q = k \cdot 2^m + 1$	Required	Multiply by k	Moduli with special structure	Very fast if q fits form

performing corrections. From the algorithm's computational process, it can be seen that the final result is not $AB \bmod q$, but rather $AB(-2^{-2n}) \bmod q$. It means that post-processing of the computation results is required to obtain the correct result.

III. IMPROVED PLANTARD ALGORITHM AND HARDWARE IMPLEMENTATION

On the basis of the introduction to the original Plantard modular reduction algorithm in the Section II-D, in this section, three improved Plantard modular multiplication algorithms are proposed for the NTT in LBC schemes. (The algorithm and the related proofs in this section are all carried out under the ML-KEM parameter set, with similar methods applicable under the ML-DSA parameter set.)

A. Improved Plantard Algorithm

1) **ShiftAdd-Plantard Algorithm:** We propose the ShiftAdd-Plantard Algorithm, an adaptation of Algorithm 1 for the ML-KEM parameter set, as shown in Algorithm 2. This will serve as the foundation and reference for the other two optimized hardware algorithms.

Algorithm 2 ShiftAdd-Plantard Algorithm

Input: A, B , with $0 \leq A, B \leq q$, $n = 13, q = 3329$
 $R = q^{-1} \bmod 2^{2n} = 61403905$

Output: $C = AB(-2^{-2n}) \bmod q$, $0 \leq C < q$

- 1: $t \leftarrow AB \bmod 2^{2n}$
- 2: $C_0 \leftarrow t_{[17:0]} + t_{[13:0]} \lll 4$ $\triangleright$ Steps 2-5: $AB \times R$
- 3: $C_1 \leftarrow t_{[11:0]} + t_{[10:0]} \lll 1$
- 4: $C_2 \leftarrow t_{[0]} + t_{[12:8]} + t_{[4:0]} + t_{[3:0]}$
- 5: $C_3 \leftarrow \{t_{[1:0]}, t_{[23:13]}\}$
- 6: $C_4 \leftarrow (C_0[17:5] + C_0[16:4] + C_0[6:0] \lll 6 + C_0[4:0] \lll$
- 7: $8 + C_1 \lll 1 + C_2[6:5] + C_3[12:0] + 1$ $\triangleright \lfloor \frac{C_0}{2^n} \rfloor + 1$
- 8: $C \leftarrow C_4 \lll 3 + C_4 \lll 2 + C_4 + C_4 \ggg 8$ $\triangleright \lfloor \frac{C_1 q}{2^n} \rfloor$
- 9: **return** C

Observing several modular reduction algorithms in [8], [17], [18], it is evident that many of these algorithms include constant multiplication operations. Specifically, the Plantard algorithm requires two constant multiplications: one by R and another by q . In hardware implementations, when the constants have relatively small bit widths, it is often feasible to decompose them into sums of powers of two, allowing the multiplication to be replaced with more efficient shift-and-add operations. For the ML-KEM parameter set, the constants are given

by: $R = 61403905 = (11101010001111001100000001)_2$ and $q = 3329 = (110100000001)_2$.

For the multiplication by R , directly computing $AB \times R \bmod 2^{2n}$ can be costly in hardware. Although R is a constant, its 26-bit length implies that the corresponding shift-and-add operations still introduce significant overhead and delay. Therefore, additional optimization is necessary to reduce the computational complexity of this step. According to steps 1-2 in Algorithm 1, we can see that C_1 actually takes bits 13–25 of the product ABR . Since the lower 8 bits do not generate any carries, they can be omitted. As a result, the partial products to be summed are shown in Figure 1. The result of $\frac{AB \times R \bmod 2^{2n}}{2^n}$ should therefore include the sum of the partial products within the purple box (corresponding to bits 13–25 in Fig. 1), along with the carry propagated from the lower bits in the blue box.

To improve the efficiency of constant multiplication, we utilize a method called grouped partial product summation, as illustrated in Fig.1. Partial products with the same color are grouped together and summed accordingly. For example, in the diagram, the four groups of partial products—green, yellow, pink, and orange—can be handled by computing the longest group (green), which involves the sum of t and $t \lll 4$. The results of the other groups can then be derived through appropriate bit-shifting and truncation. The term C_2 represents the carries generated from bits 8 to 12, while C_3 includes the remaining partial products, which can be concatenated at the bit level. Finally, Step 6 completes the summation of all partial products, truncates the lower 13 bits, and adds 1 to obtain the final result.

In addition, we note that after multiplying by q , a right shift of n bits is required. By combining this with the shift-and-add method, we apply a calculation process adjustment, as outlined in Step 7 of the Algorithm 2, which replaces Step 3 of Algorithm 1. It extends the decimal digits and performs addition to achieve carry compensation, resulting in the correct outcome. Taking ML-KEM $q = 3329$ as an example, which requires a right shift of 13 bits, we have $3329 = 2^{11} + 2^{10} + 2^8 + 2^0$. Thus, as stated in step 5 of Algorithm 2, we only need to extend the operand by 5 bits and then perform the addition to obtain the correct result. Experimental results show that, compared to performing $\times q$ and the shift separately, the calculation process adjustment effectively shortens the critical path and reduces the use of LUTs.

As mentioned in Section II-D, the results produced by Algorithm 2 are multiplied by 2^{2n} . To obtain the correct result, a method similar to the K²RED algorithm can be used, either by preprocessing the input data or postprocessing the output results $\times (-2^{2n})$, but this will incur additional hardware overhead. Considering the specific NTT computation process,

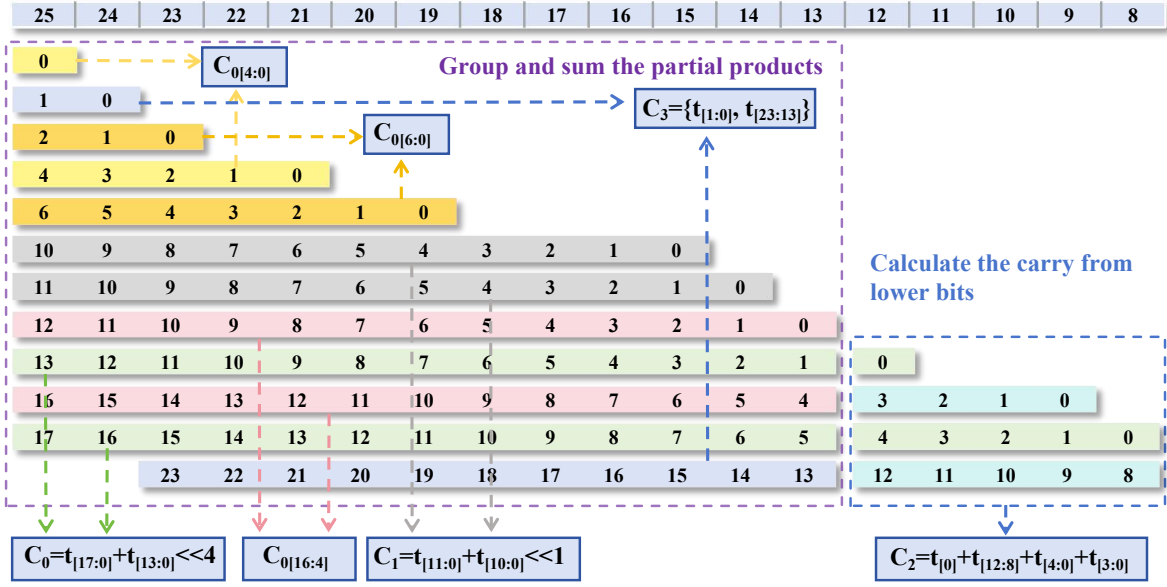


Fig. 1. The partial products and grouping of $AB \times R$.

since an INTT is required after each NTT completion, we can refer to [8] to preprocess the twiddle factors and inverse twiddle factors to obtain the correct modular multiplication results.

2) **Plantard_preR Algorithm:** Considering the disadvantages of the original algorithm, we use mathematical derivations to eliminate data preprocessing and postprocessing operations and propose the Plantard_preR algorithm, as shown in Algorithm 3. In the algorithm, to eliminate $\times(-2^{2n})$, we rewrite R as R' .

Algorithm 3 Plantard_preR Algorithm

Input: A, B , with $0 \leq A, B \leq q$, $n = 17$, $q = 3329$
 $R' = (-2^{2n} \bmod q) * R \bmod 2^{2n} = 5160670$

Output: $C = AB \bmod q$ with $0 \leq C < q$

- 1: $C_0 \leftarrow ABR' \bmod 2^{2n}$
- 2: $C_1 \leftarrow \lfloor \frac{C_0}{2^n} \rfloor + 1$
- 3: $C \leftarrow \lfloor \frac{C_1 q}{2^n} \rfloor$
- 4: **return** C

It can be expected that, compared to Algorithm 2, Plantard_preR can directly obtain the modular multiplication result, similar to the Barrett algorithm. However, as derived mathematically, this approach requires a certain degree of expansion in n . For example, in ML-KEM, n must be at least 17 to ensure that, when

$$R' = (-2^{2n} \bmod q) \times R \bmod 2^{2n},$$

the output result is directly $C = A \times B \bmod q$. As n increases, the bit-width of the operands involved in the computation also increases, leading to higher resource consumption. Nevertheless, an important advantage of this method is that it does not modify the input data and does not require any correction of

the result, making it more flexible and suitable for modular reduction operations beyond NTT-based applications.

3) **Plantard_preω Algorithm:** To further optimize the Plantard algorithm, we consider utilizing the twiddle factors in NTT to modify the original algorithm. As mentioned in [14], in NTT applications where variables are only multiplied by constants (ω), we can obtain the correct result by preprocessing the twiddle factors. This inspires us to perform certain modular multiplication steps such as $\times R$ by preprocessing the twiddle factors. Therefore, through mathematical derivations, we optimize the original algorithm by incorporating $\times R$ and $\times(-2^{2n})$ into the preprocessing of the twiddle factors. Furthermore, our derivations show that the bit-width of the twiddle factors must be increased to ensure computational correctness (The detailed proof of the algorithm will be introduced in the next subsection). Then the entire algorithm involves only $\times q$, along with shifts and a few additions, significantly simplifying the modular multiplication process. The mathematical process of the proposed Plantard_preω algorithm is shown in Algorithm 4. Due to the requirement to preprocess one of the operands, the original B in algorithm will be replaced with the twiddle factor ω in NTT.

Algorithm 4 Plantard_preω Algorithm

Input: A with $0 \leq A \leq q$, $\omega, n = 13$, $q = 3329$,

$$\tilde{\omega} = \left\lfloor \frac{\omega \cdot 2^{2n}}{q} \right\rfloor \bmod 2^{2n}$$

Output: $C = A\omega \bmod q$ with $0 \leq C < q$

- 1: $C_0 \leftarrow A\tilde{\omega} \bmod 2^{2n}$
- 2: $C_1 \leftarrow \lfloor \frac{C_0}{2^n} \rfloor + 1$
- 3: $C \leftarrow \lfloor \frac{C_1 q}{2^n} \rfloor$
- 4: **return** C

In conclusion, the three proposed algorithms cater to different hardware constraints and application scenarios. The

ShiftAdd-Plantard Algorithm is the most versatile; despite requiring post-processing, it imposes no input restrictions, making it the mandatory choice for general non-constant multiplication tasks such as Point-wise Multiplication. In contrast, Plantard_pre ω Algorithm is the superior choice for operations involving constant operands, such as the core polynomial multiplication in NTT. By modifying ω , it simplifies computational steps and eliminates pre-processing, thereby significantly reducing hardware overhead (LUTs, FFs, and DSPs) and improving efficiency compared to the original Plantard algorithm. Meanwhile, Plantard_pre R Algorithm remains unsuitable for FPGA-based designs due to the substantial overhead from larger bit widths, though it theoretically offers versatility without result correction if implementation barriers are overcome. Although demonstrated here using ML-KEM parameters, these hardware optimization techniques are adaptable to ML-DSA and other moduli with appropriate algorithmic adjustments. Furthermore, these reduction strategies are theoretically feasible for extension to larger 32- to 64-bit moduli, such as those used in FHE schemes. While accommodating larger operands would necessitate hardware modifications like wider data paths, the core optimization concepts—specifically minimizing critical-path operations and reducing memory overhead—remain broadly applicable and constitute a valuable direction for our future research.

Therefore, in the following sections of the paper, we will conduct a hardware implementation comparison of Algorithm 2 and Algorithm 4 with other modular reduction algorithms and apply them to the NTT for hardware implementation and optimization. The optimized methods we proposed are still applicable to the ML-DSA or other parameters, but adjustments are needed for n and the constant multiplication based on the modulus.

B. Correctness of the Plantard_pre ω Algorithm

Since both Algorithm 3 and Algorithm 4 are derived from modifications of the original algorithm, we present the detailed proof for Algorithm 4 in this subsection, while the proof of Algorithm 3 is provided in the Appendix. To make the algorithm more general, the constant operand will still be referred to as B instead of ω in the following proof.

Corollary 1: Let q an odd modulo with $q < \frac{2^n}{\varphi}$ ($\varphi = \frac{1+\sqrt{5}}{2}$), then Algorithm 4 is correct.

Proof (of Algorithm 4). From the computation process in Equation 5, we can distinctly obtain that $C \leq q$.

$$C = \left\lfloor \frac{\left(\left\lfloor \frac{A \cdot \tilde{B} \bmod 2^{2n}}{2^n} \right\rfloor + 1 \right) q}{2^n} \right\rfloor \quad (5)$$

- Firstly, we check that when $C = q$, it's obvious that $C \equiv 0 \bmod q$.
- Secondly, we examine the situation when $C < q$. We know that q is an odd modulo, $\exists R$ such that $R = q^{-1} \bmod 2^{2n}$. Now, let $\tilde{B} = B'q^{-1} \bmod 2^{2n}$ with $B' = B(-2^{2n}) \bmod q$. Let $\tilde{Q} = A\tilde{B} \bmod 2^{2n} =$

$AB'q^{-1} \bmod 2^{2n}$. Knowing that $0 \leq A \leq q-1$, $0 \leq B' \leq q-1$, then we obtain

$$\begin{aligned} \tilde{Q}q - AB' &\equiv AB'q^{-1}q \bmod 2^{2n} - AB' \\ &\equiv AB' - AB' \bmod 2^{2n} \equiv 0 \bmod 2^{2n} \end{aligned}$$

Consequently, we have

$$\tilde{Q}q - AB' = t \cdot 2^{2n} \Leftrightarrow t = \frac{\tilde{Q}q - AB'}{2^{2n}} \quad (6)$$

with $t \in [0, q-1]$. Also,

$$\begin{aligned} (\tilde{Q}q \cdot 2^{-2n} - AB' \cdot 2^{-2n}) \bmod q &= t \bmod q \\ \Leftrightarrow -AB' \cdot 2^{-2n} \bmod q &= t \\ \Leftrightarrow -AB' \cdot 2^{-2n} \bmod q &= \frac{\tilde{Q}q - AB'}{2^{2n}} \quad (7) \\ \Leftrightarrow AB \bmod q &= \frac{\tilde{Q}q - AB'}{2^{2n}} \end{aligned}$$

Let $\tilde{Q}_1 = \left\lfloor \frac{\tilde{Q}}{2^n} \right\rfloor$ and $Q_0 = \tilde{Q} - 2^n \tilde{Q}_1$ with $0 \leq Q_0 < 2^n$. Now, we need to prove that

$$\frac{\tilde{Q}q - AB'}{2^{2n}} = \left\lfloor \frac{(\tilde{Q}_1 + 1)q}{2^n} \right\rfloor \quad (8)$$

By $0 \leq Q_0 < 2^n$, we can obtain that

$$0 < \frac{q \cdot 2^n - Q_0q + AB'}{2^{2n}} < \frac{q \cdot 2^n + q^2}{2^{2n}} < \frac{\frac{2^{2n}}{\varphi} + \frac{2^{2n}}{\varphi^2}}{2^{2n}} = 1$$

Consequently, we have

$$\begin{aligned} \frac{\tilde{Q}q - AB'}{2^{2n}} &= \left\lfloor \frac{\tilde{Q}q - AB'}{2^{2n}} + \frac{q \cdot 2^n - Q_0q + AB'}{2^{2n}} \right\rfloor \\ &= \left\lfloor \frac{(\tilde{Q}_1 + 1)q}{2^n} \right\rfloor \end{aligned}$$

Therefore we obtain $\tilde{Q}_1 = \left\lfloor \frac{\tilde{Q}}{2^n} \right\rfloor = \left\lfloor \frac{A\tilde{B} \bmod 2^{2n}}{2^n} \right\rfloor$ and by Equation 7, we obtain that

$$AB \bmod q = \frac{\tilde{Q}q - AB'}{2^{2n}} = \left\lfloor \frac{\left(\left\lfloor \frac{A\tilde{B} \bmod 2^{2n}}{2^n} \right\rfloor + 1 \right) q}{2^n} \right\rfloor$$

Lemma 1 (of $\tilde{B}$). In addition, there is also a proof regarding our input $\tilde{B}$ in Algorithm 4 as follows:

$$\begin{aligned} \tilde{B} &= (B \cdot (-2^{2n}) \bmod q) \cdot q^{-1} \bmod 2^{2n} \\ &= (B \cdot (-2^{2n}) - \left\lfloor \frac{B \cdot (-2^{2n})}{q} \right\rfloor \cdot q) \cdot q^{-1} \bmod 2^{2n} \\ &= \left\lfloor \frac{B \cdot 2^{2n}}{q} \right\rfloor \bmod 2^{2n} \end{aligned}$$

So far, the Plantard_pre ω algorithm we proposed has been theoretically proven.

IV. DESIGN AND HARDWARE IMPLEMENTATION

In this section, we propose and implement two optimized modular multiplication units based on Algorithm 2 and Algorithm 4 presented earlier. To demonstrate the versatility of our approach, we further develop two NTT architectures: a BRAM-free iterative NTT and a Multi-path Delay Commutator (MDC) pipelined NTT, applying the Plantard_pre ω algorithm for ML-KEM and the ShiftAdd-Plantard algorithm for ML-DSA. This dual-architecture implementation effectively showcases the adaptability of our method across different LBC standards.

A. Optimized Modular Multiplication Hardware Design

Identifying the critical path of the circuit and performing reasonable partitioning and scheduling has always been crucial in a hardware design. For the design of the modular multiplication unit, it means locating registers and dividing the pipeline within limited computation cycles. In addition, to show extensibility of the algorithm proposed in this paper, we implement modular multiplication units in hardware for ML-KEM and ML-DSA to compare and evaluate performance. As the design and optimization for ML-DSA are similar to ML-KEM, we will use ML-KEM as an example for brevity.

The hardware architecture based on Algorithm 2 is shown in Fig.2. The entire modular multiplication process takes four cycles. In these four cycles, the tasks of $A \times B$ (1 clock count (cc)), $AB \times R$ constant multiplication (2cc) and $\times q$ (1cc) operation are performed, respectively. This circuit design fully implements pipelined parallelism by replacing traditional constant multiplication with shift-add and calculation process adjustment, thereby saving two DSP units and effectively shortening the critical path. Experimental results show that, compared to the original Algorithm 1, which performs addition before shifting in Step 3, applying the calculation process adjustment to implement the operation $\times q$ reduces LUT usage by 7 and improves frequency by 5.6%.

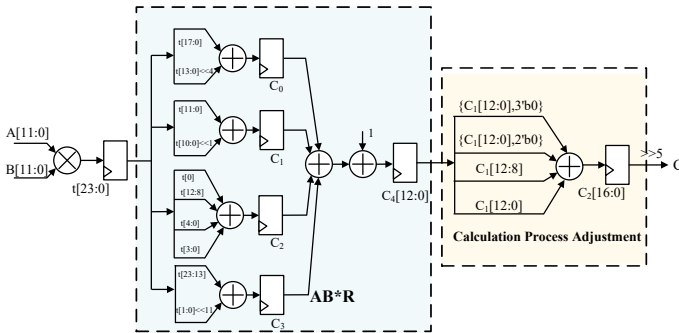


Fig. 2. Architecture of ShiftAdd-Plantard Algorithm

For Algorithm 4, the architecture of high-speed modular multiplication unit is illustrated in Fig.3. Compared to Fig.2, the bit width of the twiddle factors has increased, which means that we shift more computational burden onto the DSP. Experiments demonstrated that although the increased bit-width nearly doubles the resource consumption for twiddle factor storage, it still accounts for only a very small fraction

of the overall NTT resource footprint. For instance, under ML-KEM parameters, expanding the bit-width from 128×12 bits to 128×26 bits results in a Distributed ROM increase of only 28 LUTs and 14 FFs, which is a highly acceptable trade-off for the significant performance gains achieved. It is meaningful because our design fully utilizes the 18×25 -bit capacity of the DSP48E1, maximizing hardware efficiency compared to the underutilized resources of a standard 12×12 -bit multiplication. In addition, to match the computation bit width of DSP48E1, we depart $\tilde{\omega}$ into $\tilde{\omega}_{[25]}$ and $\tilde{\omega}_{[24:0]}$ for multiplication to fit the computational bit width of DSP48E1. For $\times q$, the calculation process adjustment in Algorithm 2 is still used to shorten the critical path. By applying the optimized Plantard_pre ω algorithm, the increase in the bit width of the twiddle factor allows full utilization of DSP48E1 on FPGA. And the cycles originally used for $AB \times R$ are reallocated to $\times q$, effectively shortening the critical path of the computation. Although the Plantard_pre ω algorithm has a much simpler computation process compared to ShiftAdd-Plantard, its bit expansion of ω under larger moduli results in higher DSP resource usage. Therefore, in our NTT design, we adopt the Plantard_pre ω algorithm for ML-KEM parameters and the ShiftAdd-Plantard algorithm for ML-DSA parameters.

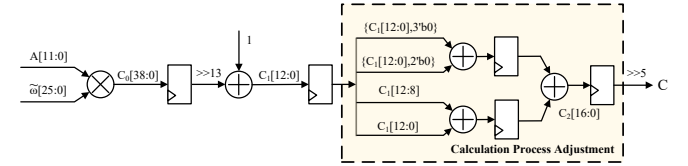


Fig. 3. Architecture of Plantard_pre ω Algorithm

B. Optimized NTT Hardware Design

The key to NTT design lies in the trade-off between resources and performance. In this work, we present distinct hardware implementations for ML-KEM and ML-DSA respectively. After analyzing and designing an efficient modular multiplication unit, it needs to be integrated into the NTT circuit architecture. This paper designs both a BRAM-free iterative NTT and an MDC pipelined NTT to provide a more comprehensive evaluation of the proposed algorithms.

1) **Iterative NTT:** An iterative NTT typically reuses one or more butterfly units to perform multi-stage operations. Each stage requires memory to store intermediate results, which are used in the following stage. The main challenge is how to efficiently organize memory and control logic to minimize computation cycles and resource on FPGA.

Current iterative NTT designs, such as [6], [7], [10], more use dual-port BRAM to access data for write and read paths of Butterfly (BF) unit. The NTT process is completed by adjusting the read/write addresses of BRAM to satisfy the computational patterns. Due to the fixed position of BRAM units on FPGA, the BRAM-based iterative NTT designs often result in significant net delays when the same BF unit needs to be repeatedly accessed, limiting the overall frequency. In 2023, Ni *et al.* proposed a BRAM-free NTT design based on FIFO [12], which leverages the flexibility and configurability

of FIFO to achieve higher frequency in FPGA implementations. In addition, this design replaces the complex address translation in BRAM with the switching of multiple FIFOs, making it more lightweight and better suited for deployment on small IoT devices.

Combined with our Plantard_prew algorithm, the BRAM-free iterative NTT structure designed in this paper is shown in Fig. 4. It consists of three main parts: Control Unit, Memory Block (three 64×24 FIFOs, two 128×26 Distributed ROMs for NTT/INTT twiddle factors), BF unit (Cooley-Tukey (CT) and Gentleman-Sande (GS) structure are both shown in Fig. 4). Comparing with other NTT iterative designs based on BRAM, FIFO-based NTT architectures achieve number of NTT stage transformations by adjusting the read and write order of three FIFOs as shown in Fig. 5.

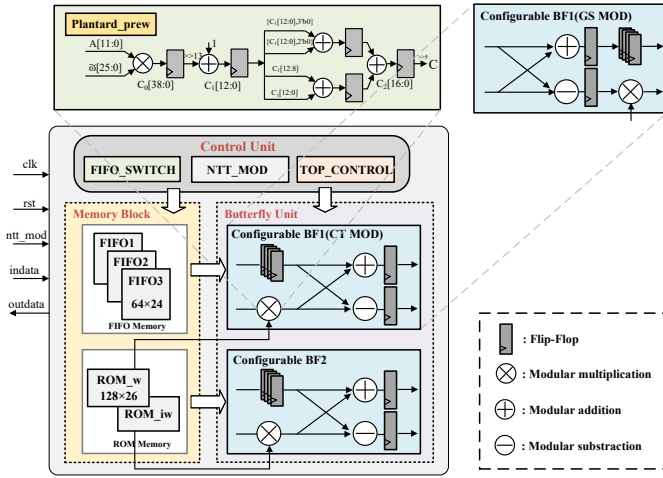
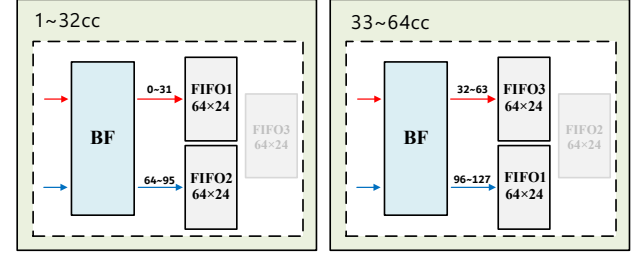


Fig. 4. Architecture of BRAM-free Iterative NTT based on Plantard_prew($q=3329$).

This design uses the modular multiplication unit shown in Fig. 3 of the previous section. Since the critical path of the proposed Plantard_prew modular multiplication is shorter than that of other reduction methods, the iterative NTT with the proposed modular multiplication achieves a higher frequency and, consequently, higher speed. Additionally, as shown in Fig. 2, our iterative NTT design uses a dual-BF approach to complete the computation of two polynomials concurrently. This design choice is based on two main considerations: (i) The iterative NTT adopts a dual-BF structure and stores polynomials in high and low bits, which allows for the simultaneous processing of two odd/even polynomials during the NTT operation in ML-KEM. Compared to using two single-BF NTTs, this approach significantly reduces the top-level control logic. (ii) Most current iterative NTT designs are based on dual-BF structures, and it has been proven that the dual-BF design can meet the parallel processing requirements of polynomials calculation in ML-KEM. Using the dual-BF design also facilitates a fair comparison among different designs.

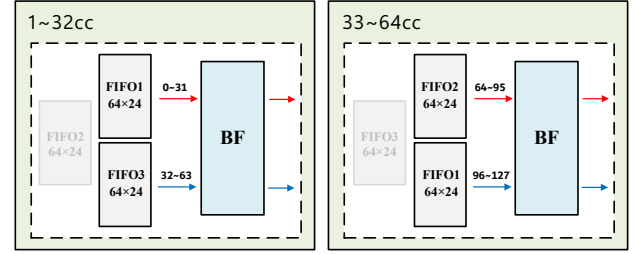
Since the Plantard_prew algorithm requires the modification of twiddle factors, it is unsuitable for PWM operations. While the proposed ShiftAdd-Plantard Algorithm could be employed. We prioritize a design featuring a unified NTT/INTT unit separated from the PWM logic. This architecture ensures

FIFO WRITE



(a) Writing Data from BF Output to FIFO (1st stage)

FIFO READ



(b) Reading Data from FIFO to BF Input (2nd stage)

Fig. 5. Data Transfer Between BF and FIFO

a higher overall system frequency and allows PWM and NTT/INTT tasks to be processed in parallel by dedicated hardware units.

2) **Pipelined NTT**: The design of the MDC pipelined NTT adjusts the BF group lengths at different stages by utilizing shift memory to temporarily store and pipeline data, thereby matching the delays between the upper and lower paths. The structural diagram of the MDC pipelined NTT (128 points) is shown in the Fig. 6. The black lines represent the NTT-CT mode connections, while the gray lines represent the INTT-GS mode connections. The key idea of pipelined NTT lies in progressively expanding multi-stage butterfly operations to ensure data pipelining.

To enable switching between different NTT modes (GS/CT), the delay units need to be appropriately allocated. When switching from NTT mode to INTT mode, the positions of the add/subtract units and the multiplication units need to be swapped, and the positions of the delay units in the two modes are exactly opposite. For instance, in NTT mode of ML-KEM, the delays for the upper and lower paths in the seven stages are $\{32, 16, 8, 4, 2, 1, 0\}$ in sequence; whereas in INTT mode, they are $\{0, 1, 2, 4, 8, 16, 32\}$. Currently, reconfigurable pipelined NTT designs allocate separate delay units for NTT and INTT modes, leading to resource redundancy and inefficient storage reuse.

To address this issue, we propose an MDC pipelined NTT design that partitions storage and computing units, as shown in Fig. 7. The proposed pipelined NTT mainly consists of four parts: Control Unit, Memory Block (SHIFT Memory and ROM Memory), Computational Unit (BF and Modular Multiplication) and C2 (Two-channel Delay Commutator). In our design, we propose a method of partitioning memory and com-

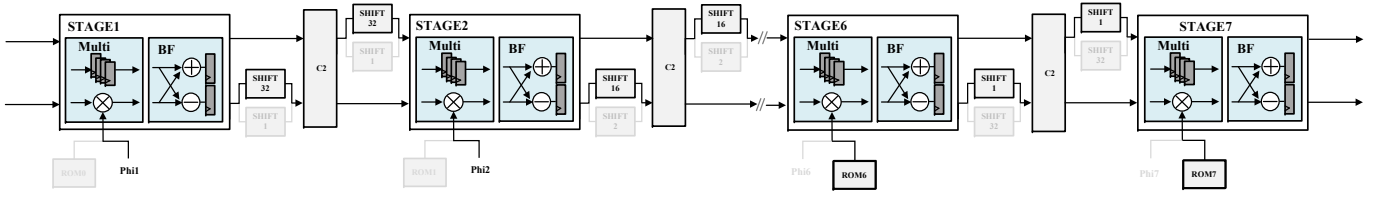


Fig. 6. The structural diagram of MDC NTT in ML-KEM (CT, $q=3329$)

TABLE II
RESULTS AND COMPARISON OF MODULAR MULTIPLICATION UNIT ON FPGA (ARTIX-7)

Designs	Algorithm	Protocol	LUT	FF	DSP	Freq.(MHz)	Slices	ENS	Cycles	Delay(ns)	CPD(ns)**	Throughput [†]
ISCAS 2023 [12]	Barrett	ML-KEM	79	54	1	294	27*	127	4	13.60	3.40	0.58
TCHES 2021 [7]	Barrett	ML-KEM	93	64	1	251	38	138	4	15.94	3.98	0.46
APCCAS 2022 [20]	K ² RED	ML-KEM	92	70	1	278	45	145	4	14.39	3.60	0.48
ShiftAdd Plantard	Plantard	ML-KEM	102	60	1	323	33	133	4	12.38	3.01	0.61
Plantard_pre _w	Plantard	ML-KEM	34	66	1	499	21	121	4	8.01	2.00	1.03
VLSI 2022 [21]	Barrett	ML-DSA	192	379	2	297	48	248	9	30.30	3.37	0.13
APCCAS 2022 [20]	K ² RED	ML-DSA	151	177	2	270	38	238	7	25.91	3.70	0.16
ShiftAdd Plantard	Plantard	ML-DSA	121	151	2	333	31	249	6	18.02	3.33	0.22
Plantard_pre _w	Plantard	ML-DSA	41	48	3	303	10	310	4	13.18	3.30	0.24

[†] Throughput is measured in $\times 10^6$ operations per Equivalent Number of Slices (ops/ENS).

* The slice of FPGA implementations is estimated by formula LUTs/4 as one slice consists 4 LUTs in Artix-7.

** Critical Path Delay (CPD).

putational units to enhance resource reuse, facilitate parameter expansion, and support reconfigurable design. Combined with our proposed ShiftAdd-Plantard and Plantard_pre_w algorithm, it significantly enhances the flexibility and overall performance of the pipelined NTT. The Computational Unit mainly consists of seven identical circuits with separate BF and Modular Multiplication modules, enabling CT/GS butterfly operations in different NTT modes by simply reordering the connections between the BF and Multiplication modules. Besides, the seven stages of BF operations share a similar circuit structure, which means that our proposed low-resource Plantard modular multiplication can save more FPGA hardware resources in the multi-stage pipelined NTT architecture. The Memory Block mainly consists of shift registers used for delays and ROMs that store the twiddle factors. ROM0-3 and ROM4-7 store the twiddle factors used for NTT and INTT respectively and the number of twiddle factors (also the depth of ROMs) in each stage of the NTT decreases in powers of 2 (64, 32, ..., 2, 1), while in the INTT, it increases. Since Xilinx ROM IP has a minimum depth of 16 [19], LUTs are used to directly store the twiddle factors for stages with fewer than 8 factors, thereby reducing ROM usage. Additionally, by managing all the storage units in a unified manner instead of distributing them across each stage, we can easily reuse the same set of delay units in NTT and INTT operations, cutting the shift register usage by half. The connection between the first and second stages of the delay commutator is shown in the Fig. 8. Since the length of BF group in the second stage is 32, the switching connection needs to change after 32 clock counts. At each stage, the connection alternates between two configurations based on the BF group length pattern to complete the complex NTT operation.

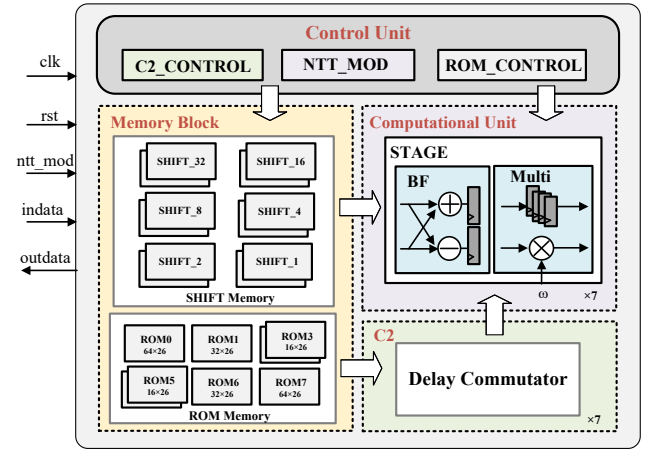


Fig. 7. Architecture of MDC pipelined NTT($q=3329$)

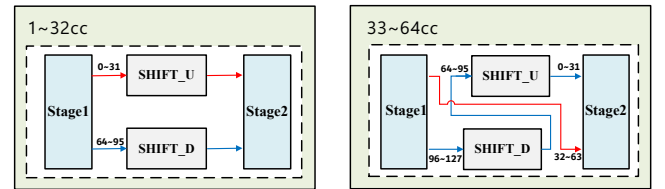


Fig. 8. The connection between the first and second stages of the delay commutator

V. RESULTS AND COMPARISONS

The proposed modular multiplication and NTT hardware architecture are implemented on the Xilinx Artix-7 Platform with Xilinx Vivado 2023.2. All implemented results have been obtained after synthesis, place-and-route with default settings.

TABLE III
THE HARDWARE DESIGN PARAMETERS OF NTT

Designs	Points	Modulus	Stages	DSP	FIFO (<i>depth</i> $\times$ <i>width</i>)	SHIFT (<i>depth</i> $\times$ <i>width</i>)	ROMs	T (<i>Theoretical</i>)
Iterative NTT	128	12	7	2	$64 \times 24 \times 3$	-	2	$448 + t_{mul} + t_{add/sub}$
	256	23	8	2	$64 \times 46 \times 3$	-	2	$1024 + t_{mul} + t_{add/sub}$
MDC Pipeline NTT	128	12	7	7	-	$2^i \times 12 \times 2, i = 0, 1, \dots, 5$	2	$128 + 7 \times (t_{mul}t_{add/sub})$
	256	23	8	16	-	$2^i \times 23 \times 2, i = 0, 1, \dots, 6$	2	$256 + 8 \times (t_{mul}t_{add/sub})$

A. Modular Multiplication

In order to make a fair comparison, we replicate the Barrett, KRED, and Plantard algorithms on the same platform, and unified the modular multiplication operation cycle to 4 clock counts (only in the same number of cycles, comparison of frequencies is meaningful). For ML-KEM, as shown in Table II, design based on Plantard_prew consumes 34 LUTs, 66 FFs, 21 slices with an operating frequency of 499 MHz. Compared with other algorithm (including original Plantard in Algorithm 2), our design reduced LUT by 57 – 67% and slices by 22 – 53%. In four computational cycles, our optimized design increased operating frequency by 54.5 – 98% and complete modular multiplication in only 8.01ns, resulting in a significantly higher throughput of 1.03×10^6 operations per second. For ML-DSA, the ShiftAdd-Plantard algorithm reduces LUT usage by 20 – 37%, FF usage by 15 – 60%, and delay by 30.5 – 40.5% compared to other algorithms, achieving a throughput of 0.22×10^6 operations per second, highlighting the advantages of our approach. However, for Plantard_prew algorithm, the increased operand width expands the pre-computed twiddle factors from 23 bits to 48 bits, requiring more DSPs for multiplication. Although LUT, FF, and delay are significantly reduced compared to other designs. Therefore, in our NTT design, we adopt the Plantard_prew algorithm for ML-KEM parameters and the ShiftAdd-Plantard algorithm for ML-DSA parameters.

Besides, in the overall NTT design, the critical path is influenced by the top-level control logic and routing, making it difficult to achieve such a high frequency as in a single modular multiplication unit. However, the experimental results still fully demonstrate that the proposed ShiftAdd-Plantard and Plantard_prew modular multiplication circuit has a shorter critical path compared to other algorithms, providing more timing margin for automated routing in the overall NTT implementation.

B. NTT

1) **Hardware Metrics and Parameters:** Before evaluating the performance and resource of NTT, we need to establish a clear mapping and understanding from theory and algorithm to the specific resources involved in the FPGA hardware of the NTT. For lattice-based PQC protocols ML-KEM and ML-DSA, we list the relevant parameters of the two NTT designs, FPGA reference resource, and theoretical computation cycles in the Table III.

- DSP:** The number of NTT stages determines the DSP count in the design. For iterative NTT, despite some using higher radix butterfly units, DSP usage typically matches the number of radix-2 units. In pipelined NTT without pre/post-processing, one DSP per stage suffices, so DSP count roughly equals the number of stages. Since DSP48E1 handles 18×25 -bit multiplication, larger bit-widths require multiple DSPs. For ML-DSA parameters, the DSP count doubles, including in iterative NTT.
- Storage Memory:** The number of BRAMs in NTT depends on the butterfly operation method. Iterative NTT requires storage for intermediate results, so most designs use BRAMs. As noted in Section IV-B1, our iterative NTT employs a FIFO-based approach, enhancing flexibility and frequency. Pipelined NTT does not use BRAMs but requires shift registers for buffering, contributing to its higher resource overhead compared to iterative NTT.
- Latency:** According to NTT principles, the number of BF operations is fixed, so the computation cycle has a theoretical minimum, defined as the total cycles from first input to last output. For the same parameters, an NTT design with a cycle count closer to this minimum is better. Let n be the NTT size, t_{mul} the cycles per modular multiplication, and $t_{add/sub}$ the cycles per modular addition/subtraction. In iterative NTT, each stage has $n/2$ BF operations, and there are $\log_2 n$ stages. Assuming continuous BF operations, the theoretical computation cycle is:

$$T_{iterative} = \frac{n}{2} \log_2 n + t_{mul} + t_{add/sub}$$

For MDC Pipelined NTT, data input requires $n/2$ cycles, each stage of butterfly operation takes $t_{mul} + t_{add/sub}$, and the data transfer delay between stages is $n/2 - 1$ cycles. Therefore, the theoretical computation cycle for MDC pipelined NTT is:

$$T_{pipelined} = n - 1 + \log_2 n \times (t_{mul} + t_{add/sub})$$

- Resource and performance evaluation parameters:** In order to standardize FPGA resources on-chip, the parameter ENS (Equivalent Number of Slices) is generally used to convert various resources into equivalent Slices for comparison. The formula is:

$$ENS = \frac{LUTs}{4} + \frac{FFs}{8} + BRAMs \times 200 + DSPs \times 100$$

TABLE IV
RESULTS AND COMPARISON OF NTT ON FPGA WITH EFFICIENCY METRICS

Designs	NTT Type	Platform	$n/\log_2 q$	LUT/FF/DSP/BRAMs	Freq.(MHz)	ENS	Cycles	Delay(μ s)	ATP	Throughput [†]
*TCHES 2021 [7]	Iterative	Artix-7	128/12	1579/1058/2/3	161	1327	512	3.18	4.22	0.237
ARITH 2021 [8]	Iterative	Artix-7	128/12	810/717/4/2	222	1092	324	1.46	1.59	0.627
*ISCAS 2021 [9]	Iterative	Artix-7	128/12	609/640/2/4	257	1232	490	1.90	2.35	0.427
*ISCAS 2023 [12]	Iterative	Artix-7	128/12	1154/1031/2/0	300	617	456	1.52	0.94	1.066
*TCAS-I 2024 [22]	Iterative	Artix-7	128/12	1005/599/2/0	227	526	448	1.97	1.04	0.966
TCAS-I 2025 [23]	Iterative-pipeline	Artix-7	128/12	1300/800/4/0	234	825	224	0.96	0.792	1.263
*TCAS-I 2025 [24]	Iterative	Artix-7	128/12	1452/1278/4/2.5	345	1423	235	0.68	0.968	1.471
*TC 2023 [25]	Iterative	Artix-7	128/12	784/441/2/3	200	1051	313	1.57	1.65	0.606
Electronics 2025 [26]	Iterative	Virtex-7	128/12	6841/6982/0/0	249	2583	898	3.60	9.32	0.108
*SOCC 2025 [27]	Iterative	Artix-7	256/23	3101/655/6/3	250	1423	1280	5.12	10.53	0.137
*TCHES 2022 [28]	Iterative	Artix-7	256/23	1919/1301/8/2	97	1672	256**	2.64	4.41	0.227
Our Iterative NTT	Iterative	Artix-7	128/12	803/895/2/0	278	513	457	1.64	0.84	1.189
			256/23	2038/1452/4/0	238	1091	1034	4.34	4.74	0.211
		Virtex-7	128/12	853/907/2/0	293	527	457	1.56	0.82	1.217
			256/23	2113/1463/4/0	244	1111	1034	4.24	4.71	0.212
VLSI 2024 [29]	SDF Pipeline	Virtex-7	256/16	1793/2047/8/1.5	322	1804	318	0.99	1.78	0.560
TCAS-II 2022 [30]	Pipeline	Virtex-7	256/12	1362/1036/7/0.5	235	1270	556	2.37	3.00	0.332
			256/16	2294/1645/7/3	228	2079	556	2.44	5.07	0.197
HOST 2024 [31]	MDC Pipeline	Artix-7	256/23	3441/1859/16/0	119	2460	256**	2.15	5.29	0.189
Our Pipelined NTT	MDC Pipeline	Artix-7	1919/1615/7/0	128/12	232	1382	169	0.73	1.00	0.991
			256/23	3412/4877/16/0	204	3062	335	1.64	5.03	0.199
		Virtex-7	128/12	1924/1607/7/0	303	1382	169	0.56	0.77	1.297
			256/23	3400/4875/16/0	233	3059	335	1.44	4.40	0.227

* These work support NTT/INTT/PWM, others only support NTT/INTT.

** The article does not provide the NTT computation cycles, so the minimum theoretical cycles are used here.

† Throughput is measured in $\times 10^3$ operations per ENS (ops/ENS).

To evaluate the balance between resource overhead and performance, we introduce ATP (Area Time Product) to assess the hardware efficiency of different designs. A lower ATP indicates a more efficient design. The formula is:

$$ATP = Delay \times ENS$$

Throughput quantifies the speed achieved per unit of hardware resource (ENS), where a higher value indicates a more efficient design.

$$Throughput = \frac{Throughput}{Area} = \frac{10^6}{Delay(\mu s) \times ENS}$$

This paper designs both iterative and pipelined NTT structures optimized for the ML-KEM and ML-DSA parameters. We provide a comprehensive evaluation by comparing our design against several state-of-the-art implementations, including [7]–[9], [12], [22]–[31]. The relevant parameters and performance results of this comparative analysis are summarized in Table IV.

2) **Iterative NTT Results Analysis:** The proposed BRAM-free Iterative NTT achieves an excellent balance between resource consumption and performance, leveraging optimization with the Plantard algorithm to maintain high frequency and

low latency on the Artix-7 platform. As shown in Table IV, our Iterative NTT consumes LUT/FF/DSP resources of 803/895/2 for the 128/12 parameters and 2038/1452/4 for the 256/23 parameters, achieving frequencies of 278 MHz and 238 MHz, respectively. Polynomial computations are completed in just 457cc (Theoretical cycle: 448) and 1034cc (Theoretical cycle: 1024).

Currently, there are many iterative NTT designs based on ML-KEM parameters, each focusing on different optimization aspects. The papers [7]–[9] respectively utilize Barrett, K²RED, and LUT-Based modular reduction algorithms to accelerate and optimize NTT. In comparison, our NTT design, implemented with the Plantard algorithm, achieves a frequency improvement ranging from 8.2% (compared to 257 MHz in [9]) to 72.7% (compared to 161 MHz in [7]) and has the lowest ENS among these three, fully demonstrating its advantages in terms of resource and performance. Designs in [8], [23], [24] achieve lower latency and higher throughput through an architectural trade-off: they utilize structures such as the 4-butterfly unit (which consumes 4 DSPs) to enhance parallelism. Our design, in contrast, aims for high throughput while maintaining low resource consumption utilizing only 2 DSPs. The designs in [12] and [22], like ours, all adopt a

BRAM-free architecture which features lightweight, flexibility and high speed. Our frequency (278 MHz) is higher than [22] and slightly lower than [12], but it has the lowest ATP (0.84) among these three specific designs, indicating a better balance between resource utilization and performance. Design in [25] proposed a split-radix NTT scheme that resolves the address conflict issue. However, due to the higher operating frequency of our design, it achieves a similar total delay compared to [25], while performing significantly better in terms of ATP (0.84 compared to 1.65). The design in [26] proposed an NTT structure that avoids the use of DSP units. Compared to this approach, our design achieves lower ATP and higher throughput.

There are fewer iterative ML-DSA NTT designs for reference. Article [28] proposed an iterative NTT design for ML-DSA using a 4-butterfly structure. Although the delay of our design is 64.4% higher than that of [28], it achieves 2.45 times the frequency with only half the DSP units and has nearly identical ATP. The design in [27] evaluates multiple metrics in an FPGA-based NTT hardware accelerator using ML-DSA parameters. In comparison, while our design operates at a lower frequency, it achieves significantly lower ATP and higher throughput, demonstrating a superior balance between resource usage and performance.

3) **Pipelined NTT Results Analysis:** This paper also implements the MDC Pipelined NTT based on the ShiftAdd Plantard and Plantard_prew Algorithm. By partitioning the storage and computation units, the design improves flexibility, fully reuses storage units, and allows for easy parameter scaling by adjusting the corresponding ROM and shift registers. This approach results in better performance and shorter computation cycles. On the Artix-7 platform, our Pipelined NTT consumes LUT/FF/DSP resources of 1919/1615/7 and 3412/4877/16 respectively, reaching frequencies of 232 MHz and 204 MHz. The polynomial operation is completed in only 169cc (Theoretical cycle: 128) and 335cc (Theoretical cycle: 256). To allow for a fair comparison with high-performance implementations, we also deployed the design on the Virtex-7 platform. On this platform, the proposed design achieves maximum frequencies of 303 MHz and 233 MHz, with execution delays reduced to 0.56 μ s and 1.44 μ s, respectively. Pipelined NTT designs are often applied to scenarios with higher moduli and larger point sizes, such as the NTT generators in [32] and [33]. These designs feature flexible architectures but typically require higher-performance platforms to achieve optimal results. Existing designs such as [29] and [30] are also implemented on Virtex-7 and target similar point sizes ($n = 256$), but with a smaller modulus bit-width (16-bit). Dealing with the 23-bit modulus for ML-DSA exceeds the DSP's standard computational width, introducing greater challenges in critical path dividing and timing arrangement. Despite this complexity, our design on Virtex-7 outperforms [30] in terms of frequency and significantly reduces latency. While [29] achieves a higher frequency due to its smaller 16-bit modulus, our design maintains competitive performance while supporting the larger field required by ML-DSA. The design in [31] and this design both target ML-DSA parameters and are implemented on the same Artix-7 platform. Although

the design in this paper consumes twice as many FFs in terms of resource overhead, it achieves a 71.4% higher frequency (204 MHz compared to 119 MHz). In addition, this paper reduces the delay by 23.7% and ATP by 4.9%, which fully demonstrates the compactness and superior performance of our design.

Regarding side-channel resistance, the proposed architecture inherently supports constant-time execution, providing a fundamental defense against timing attacks. Specifically, all NTT/INTT operations follow a deterministic sequence and utilize identical data paths, ensuring that the total computation cycles remain fixed and independent of input values. Furthermore, the design exhibits high structural compatibility with advanced countermeasures such as masking and hiding. The modular architecture allows for the seamless integration of lightweight random number generators or masked inputs with minimal modifications to the core data path, establishing a robust foundation for future security enhancements against power and electromagnetic analysis.

VI. CONCLUSION

This paper presents three optimized versions of the Plantard algorithm—ShiftAdd-Plantard, Plantard_prew, and Plantard_prew for LBC scheme. Through mathematical derivation, the original algorithm is optimized by applying grouped partial product summation, merging pre-processing and constant multiplication into This paper presents three optimized versions of the Plantard algorithm—ShiftAdd-Plantard, Plantard_prew, and Plantard_prew for LBC scheme. Through mathematical derivation, the original algorithm is optimized by applying grouped partial product summation, merging pre-processing and constant multiplication into the twiddle factors, and adjusting the computation process. Experimental results show that our modular multiplication design based on ShiftAdd Plantard and Plantard_prew have shorter critical paths and outperform other methods such as Barrett and KRED. Moreover, we design BRAM-free iterative and MDC-pipelined NTT architectures applying Plantard_prew algorithm for ML-KEM parameters and the ShiftAdd-Plantard algorithm for ML-DSA parameters. The iterative design reaches 278 MHz on the Artix-7, approaching the theoretical minimum operation cycles counts. It achieves the lowest ATP among iterative counterparts with minimal delay and resource usage, highlighting its lightweight and high-speed nature. The pipelined design features partitioned storage and computation units, easily extendable to other applications by adjusting the storage unit's depth and bit-width. Compared to state-of-the-art designs, our approach for ML-DSA reduces delay by 23.7% and improves ATP by 4.9%. In future work, we will apply the NTT designs to full ML-KEM and ML-DSA systems, aiming to explore more lightweight cryptographic circuit designs. We also plan to thoroughly evaluate the NTT unit's impact on overall system performance in terms of resource consumption and frequency, providing a systematic methodology for cryptographic hardware design. We also plan to investigate side-channel protections, particularly against power and electromagnetic attacks, and integrate advanced countermeasures like masking to further enhance the physical security of the system.

REFERENCES

- [1] D. Moody, "Module-lattice-based key-encapsulation mechanism standard," *National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. FIPS 203*, 2023.
- [2] G. M. Raimondo and L. E. Locascio, "Module-lattice-based digital signature standard," *FIPS 204 Standard*, 2023.
- [3] J. Aumasson, D. Bernstein, W. Beullens, C. Dobraunig, M. Eichlseder, S. Fluhrer, S. Gazdag, A. Hülsing, P. Kampanakis, S. Kölbl *et al.*, "Sphincs+-submission to the 3rd round of the nist post-quantum project v3. 1," *NIST PQC Round*, pp. 1–63, 2022.
- [4] Z. Ni, A. Khalid, M. O'Neill, W. Liu *et al.*, "Hpka: A high-performance crystals-kyber accelerator exploring efficient pipelining," *IEEE Transactions on Computers*, vol. 72, no. 12, pp. 3340–3353, 2023.
- [5] Y. Zhang, C. Wang, A. Khalid, M. O'Neill, W. Liu *et al.*, "Ultra high-speed polynomial multiplications for lattice-based cryptography on fpgas," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 4, pp. 1993–2005, 2022.
- [6] T.-H. Nguyen, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "High-speed ntt accelerator for crystal-kyber and crystal-dilithium," *IEEE Access*, 2024.
- [7] Y. Xing and S. Li, "A compact hardware implementation of cca-secure key exchange mechanism crystals-kyber on fpga," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 328–356, 2021.
- [8] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed ntt-based polynomial multiplication accelerator for post-quantum cryptography," in *2021 IEEE 28th symposium on computer arithmetic (ARITH)*. IEEE, 2021, pp. 94–101.
- [9] C. Zhang, D. Liu, X. Liu, X. Zou, G. Niu, B. Liu, and Q. Jiang, "Towards efficient hardware implementation of ntt for kyber on fpgas," in *2021 IEEE international symposium on circuits and systems (ISCAS)*. IEEE, 2021, pp. 1–5.
- [10] F. Yaman, A. C. Mert, E. Öztürk, and E. Savaş, "A hardware accelerator for polynomial multiplication operation of crystals-kyber pqc scheme," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1020–1025.
- [11] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "Cfntt: Scalable radix-2/4 ntt multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 94–126, 2022.
- [12] Z. Ni, A. Khalid, W. Liu, and M. O'Neill, "Towards a lightweight crystals-kyber in fpgas: an ultra-lightweight bram-free ntt core," in *IEEE International Symposium on Circuits and Systems 2023*. IEEE, 2023.
- [13] P. Longa and M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," in *Cryptology and Network Security*, S. Foresti and G. Persiano, Eds. Cham: Springer International Publishing, 2016, pp. 124–139.
- [14] T. Plantard, "Efficient word size modular arithmetic," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1506–1518, 2021.
- [15] J. Huang, J. Zhang, H. Zhao, Z. Liu, R. C. Cheung, Ç. K. Koç, and D. Chen, "Improved plantard arithmetic for lattice-based cryptography," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 4, pp. 614–636, 2022.
- [16] F. Can, A. Üstün, B. Örs, E. Alaybeyoğlu, and E. Savaş, "Hardware implementation of k 2 red and plantard modular multiplication algorithms in post-quantum cryptography," in *2024 Panhellenic Conference on Electronics & Telecommunications (PACET)*. IEEE, 2024, pp. 1–6.
- [17] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [18] P. Barrett, "Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor," in *Conference on the Theory and Application of Cryptographic Techniques*. Springer, 1986, pp. 311–323.
- [19] Xilinx, *Block Memory Generator v8.3: LogiCORE IP Product Guide*, Xilinx, Inc., March 2017, v8.3. [Online]. Available: https://www.xilinx.com/support/documentation/ip_documentation/blk_mem_gen/v8_3/pg058-blk-mem-gen.pdf
- [20] M. Li, J. Tian, X. Hu, Y. Cao, and Z. Wang, "High-speed and low-complexity modular reduction design for crystals-kyber," in *2022 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*. IEEE, 2022, pp. 1–5.
- [21] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient implementation of dilithium signature scheme on fpga soc platform," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 9, pp. 1158–1171, 2022.
- [22] T.-H. Nguyen, D.-T. Dam, P.-P. Duong, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "Efficient hardware implementation of the lightweight crystals-kyber," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.
- [23] T.-H. Nguyen, D.-T. Dam, P.-P. Duong, T.-K. Dang, T.-T. Hoang, and C.-K. Pham, "A timing-constrained design methodology for radix-2 k ntt in polynomial arithmetic," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.
- [24] Y. Cui, J. Chen, Z. Ni, Z. Zhang, C. Wang, and W. Liu, "Instruction-based high-performance hardware controller of crystals-kyber with balanced resource utilization," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.
- [25] W. Guo and S. Li, "Split-radix based compact hardware architecture for crystals-kyber," *IEEE Transactions on Computers*, 2023.
- [26] O. S. Sonbul, M. Rashid, and A. Y. Jaffar, "Accelerating crystals-kyber: High-speed ntt design with optimized pipelining and modular reduction," *Electronics*, vol. 14, no. 11, p. 2122, 2025.
- [27] M. Imran, A. Khalid, C. Rafferty, M. Rashid, and M. O'Neill, "Evaluating area-time-power trade-offs in ntt accelerators for post-quantum cryptography," in *38th IEEE International System-on-Chip Conference*. Institute of Electrical and Electronics Engineers Inc., 2025.
- [28] C. Zhao, N. Zhang, H. Wang, B. Yang, W. Zhu, Z. Li, M. Zhu, S. Yin, S. Wei, and L. Liu, "A compact and high-performance hardware architecture for crystals-dilithium," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 270–295, 2022.
- [29] Z. Wang, Y. Yang, J. Hou, Y. Su, and C. Yang, "A scalable and efficient ntt/intt architecture using group-based pairwise memory access and fast interstage reordering," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, pp. 1–5, 2024.
- [30] Z. Ye, R. C. C. Cheung, and K. Huang, "Pipentt: A pipelined number theoretic transform architecture," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 10, pp. 4068–4072, October 2022. [Online]. Available: <https://doi.org/10.1109/TCSII.2022.3184703>
- [31] H. Zhao, C. Zhao, W. Zhu, B. Yang, S. Wei, and L. Liu, "Sparse polynomial multiplication-based high-performance hardware implementation for crystals-dilithium," in *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2024, pp. 150–159.
- [32] Y. Yang, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, "Nttgen: a framework for generating low latency ntt implementations on fpga," in *Proceedings of the 19th ACM International Conference on Computing Frontiers*, 2022, pp. 30–39.
- [33] F. Hirner, A. C. Mert, and S. S. Roy, "Proteus: A pipelined ntt architecture generator," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2024.



Jiansheng Chen is currently working towards the Ph.D degree in Information and Communication Engineering at Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. He received the Bachelor's degree in Microelectronics Science and Engineering from NUAA, Nanjing, China, in 2023. His research interests include hardware security, cryptographic engineering, post-quantum cryptography.



Bei Wang received the B.S. degree in Mathematics from Hefei University of Technology, Hefei, China, and the Ph.D. degree in Information and Communication Engineering from University of Science and Technology of China, Hefei, China. She is currently a lecturer in the College of Integrated Circuit, NUAA. Her research interests include cryptography and quantum computing. Her primary research focus is on software and hardware acceleration of post-quantum cryptographic algorithms and analysis of quantum computing's impact on cryptographic security.



Ziying Ni is received the Ph.D. degree in Electronics and Electrical Engineering from Queen's University Belfast (QUB), Belfast, UK in 2024, and M.S. degree in Electronics and Communication Engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2021. He is currently a research fellow at Centre for Secure Information Technologies (CSIT) of QUB, Belfast, UK. His research interests include hardware security, cryptographic engineering.



Li Mengxue is currently working towards the Master's degree in Electronic Information at Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. She received the Bachelor's degree from Zhejiang Gongshang University, in 2023. Her research interests include hardware design and optimization of core operators for post-quantum cryptography.



Fei Lyu (Member, IEEE) received a B.S. degree in information engineering from Shanghai University, Shanghai, China, in 2011 and a Ph.D. degree from the School of Electronic Science and Engineering at Nanjing University, Nanjing, China, in 2017. He is currently an associate researcher with the College of Integrated Circuits, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. His research interests include computer arithmetic, CMOS sensors, and reconfigurable computing.



in hardware security.

Yijun Cui received the B.Sc. degree in Information Engineering and Ph.D. degree in Information and Communication Systems from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2010 and 2020, respectively. He was a visiting PhD student in the Data Security System Group in the Centre of Secure Information Technologies, at Queen's University Belfast, UK from 2014 to 2015. He is currently an associate professor in the College of Electronics and Information Engineering of NUAA. His current research interests are mainly



Weiqiang Liu (M'12, SM'15) received the B.Sc. degree in Information Engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China and the Ph.D. degree in Electronic Engineering from the Queens University Belfast (QUB), Belfast, UK, in 2006 and 2012, respectively. He is currently a Professor and the Vice Dean of College of Electronic and Information Engineering & College of Integrated Circuits, NUAA. He has published two research books and over 190 leading journal and conference papers (over 70 IEEE and

ACM journals including 8 invited papers). He has been awarded the prestigious Excellent Young Scholar Award by National Natural Science Foundation of China in 2020 and the Young Scientist Award by Fok Ying Tung Education Foundation, Ministry of Education, China, 2022. He has been listed in the Stanford University's 2020 list of the top 2% scientists in the world. He is the VP-Elect for TA of the IEEE Nanotechnology Council (NTC). He serves as the Associate Editors for IEEE TCAS-I, TETC and TC, the Steering Committee Member of IEEE TVLSI, the Guest Editor of Proceedings of the IEEE. He is the program co-chair of IEEE ARITH 2020 and ACM/IEEE NANOARCH 2022, and also technical program committee members for a number of IEEE/ACM conferences including DAC, DATE, ARITH, ISCAS et al. He is a tutorial organizer and speaker in DAC 2022, DATE 2022, IEEE ISCAS 2021 and COINS 2021. He is a member of IEEE NTC AdCom and CASCOM/VSA Technical Committee of IEEE CAS Society. His research interest focuses on energy efficient and secure computing integrated circuits and systems.